

usarReducer

`useReducer` Es un React Hook que te permite agregar un **reductor** a tu componente.

```
const [state, dispatch] = useReducer(reducer, initialArg, init?)
```

- Referencia

- `useReducer(reducer, initialArg, init?)`
- `dispatch` función

- Uso

- Agregar un reductor a un componente
- Escribiendo la función reductora
- Evitar recrear el estado inicial

- Solución de problemas

- He enviado una acción, pero el registro me da el valor del estado anterior.
- He enviado una acción, pero la pantalla no se actualiza.
- Una parte del estado de mi reductor se vuelve indefinida después del despacho.
- Todo el estado de mi reductor se vuelve indefinido después del despacho.
- Me aparece el error: “Demasiados renderizados”.
- Mi función reductora o inicializadora se ejecuta dos veces.

Referencia

useReducer(reducer, initialState, init?)

Llama `useReducer` al componente en el nivel superior para gestionar su estado con un [reductor](#).

```
import { useReducer } from 'react';

function reducer(state, action) {
  // ...
}

function MyComponent() {
  const [state, dispatch] = useReducer(reducer, { age: 42 });
  // ...
}
```

[Vea más ejemplos a continuación.](#)

Parámetros

- **reducer** La función reductora especifica cómo se actualiza el estado. Debe ser pura, recibir el estado y la acción como argumentos y devolver el siguiente estado. El estado y la acción pueden ser de cualquier tipo.
- **initialArg**: El valor a partir del cual se calcula el estado inicial. Puede ser un valor de cualquier tipo. La forma en que se calcula el estado inicial a partir de él depende del siguiente `init` argumento.
- **opcional init**: La función inicializadora que debe devolver el estado inicial. Si no se especifica, el estado inicial se establece en `initialArg`. De lo contrario, el estado inicial se establece en el resultado de llamar a `init(initialArg)`.

Devoluciones

`useReducer` Devuelve un array con exactamente dos valores:

1. El estado actual. Durante el primer renderizado, se establece en `init(initialArg)` o `initialArg` (si no hay `init`).
2. La [dispatch función](#) que permite actualizar el estado a un valor diferente y activar una nueva renderización.

Advertencias

- `useReducer` Es un Hook, por lo que solo puedes llamarlo **en el nivel superior de tu componente** o en tus propios Hooks. No puedes llamarlo dentro de bucles o condiciones. Si necesitas hacerlo, crea un nuevo componente y traslada el estado a él.
- La `dispatch` función tiene una identidad estable, por lo que a menudo se omite en las dependencias de Efecto, pero incluirla no provocará que el Efecto se active. Si el linter permite omitir una dependencia sin errores, es seguro hacerlo. [Obtenga más información sobre cómo eliminar las dependencias de Efecto.](#)
- In Strict Mode, React will **call your reducer and initializer twice** in order to [help you find accidental impurities](#). This is development-only behavior and does not affect production. If your reducer and initializer are pure (as they should be), this should not affect your logic. The result from one of the calls is ignored.

dispatch function

The `dispatch` function returned by `useReducer` lets you update the state to a different value and trigger a re-render. You need to pass the action as the only argument to the `dispatch` function:

```
const [state, dispatch] = useReducer(reducer, { age: 42 });

function handleClick() {
  dispatch({ type: 'incremented_age' });
  // ...
}
```

React will set the next state to the result of calling the `reducer` function you've provided with the current `state` and the action you've passed to `dispatch`.

Parameters

- `action`: The action performed by the user. It can be a value of any type. By convention, an action is usually an object with a `type` property identifying it and, optionally, other properties with additional information.

Returns

`dispatch` functions do not have a return value.

Caveats

- The `dispatch` function **only updates the state variable for the next render**. If you read the state variable after calling the `dispatch` function, **you will still get the old value** that was on the screen before your call.
- If the new value you provide is identical to the current `state`, as determined by an `Object.is` comparison, React will **skip re-rendering the component and its children**. This is an optimization. React may still need to call your component before ignoring the result, but it shouldn't affect your code.
- React **batches state updates**. It updates the screen **after all the event handlers have run** and have called their `set` functions. This prevents multiple re-renders during a single event. In the rare case that you need to force React to update the screen earlier, for example to access the DOM, you can use `flushSync`.

Usage

Adding a reducer to a component

Call `useReducer` at the top level of your component to manage state with a `reducer`.

```
import { useReducer } from 'react';

function reducer(state, action) {
  // ...
}

function MyComponent() {
  const [state, dispatch] = useReducer(reducer, { age: 42 });
  // ...
}
```

`useReducer` returns an array with exactly two items:

1. The current state of this state variable, initially set to the initial state you provided.
2. The dispatch function that lets you change it in response to interaction.

To update what's on the screen, call dispatch with an object representing what the user did, called an *action*:

```
function handleClick() {
  dispatch({ type: 'incremented_age' });
}
```

React will pass the current state and the action to your reducer function. Your reducer will calculate and return the next state. React will store that next state, render your component with it, and update the UI.

App.js

Download ↺ × ↗

```
import { useReducer } from 'react';

function reducer(state, action) {
  if (action.type === 'incremented_age') {
    return {
      age: state.age + 1
    };
  }
}
```

```
    throw Error('Unknown action.');
```

```
  }
```

▼ Show more

`useReducer` is very similar to `useState`, but it lets you move the state update logic from event handlers into a single function outside of your component. Read more about [choosing between `useState` and `useReducer`](#).

Writing the reducer function

A reducer function is declared like this:

```
function reducer(state, action) {  
  // ...  
}
```

Then you need to fill in the code that will calculate and return the next state. By convention, it is common to write it as a [switch statement](#). For each

case in the switch, calculate and return some next state.

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'incremented_age': {  
      return {  
        name: state.name,  
        age: state.age + 1  
      };  
    }  
    case 'changed_name': {  
      return {  
        name: action.nextName,  
        age: state.age  
      };  
    }  
  }  
  throw Error('Unknown action: ' + action.type);  
}
```

Actions can have any shape. By convention, it's common to pass objects with a `type` property identifying the action. It should include the minimal necessary information that the reducer needs to compute the next state.

```
function Form() {  
  const [state, dispatch] = useReducer(reducer, { name: 'Taylor', age: 42  
  
  function handleClick() {  
    dispatch({ type: 'incremented_age' });  
  }  
  
  function handleInputChange(e) {  
    dispatch({  
      type: 'changed_name',  
      nextName: e.target.value  
    });  
  }  
  // ...  
}
```

The action type names are local to your component. [Each action describes a single interaction, even if that leads to multiple changes in data.](#) The shape of the state is arbitrary, but usually it'll be an object or an array.

Read [extracting state logic into a reducer](#) to learn more.

Pitfall

State is read-only. Don't modify any objects or arrays in state:

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'incremented_age': {  
      // 🚩 Don't mutate an object in state like this:  
      state.age = state.age + 1;  
      return state;  
    }  
  }  
}
```

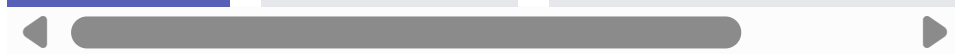
Instead, always return new objects from your reducer:


```
function reducer(state, action) {  
  switch (action.type) {  
    case 'incremented_age': {  
      // ✅ Instead, return a new object  
      return {  
        ...state,  
        age: state.age + 1  
      };  
    }  
  }  
}
```

Read [updating objects in state](#) and [updating arrays in state](#) to learn more.

Basic useReducer examples

1. Form (object) 2. Todo list (array) 3. Writing concise update log



Example 1 of 3:

Form (object)

In this example, the reducer manages a state object with two fields: `name` and `age`.

App.js

Download ↺ × ↗

```
import { useReducer } from 'react';  
  
function reducer(state, action) {  
  switch (action.type) {  
    case 'incremented_age': {  
      return {  
        name: state.name,  
        age: state.age + 1  
      }  
    }  
  }  
}
```



Next Example

React saves the initial state once and ignores it on the next renders.

```
function createInitialState(username) {  
  // ...  
}  
  
function TodoList({ username }) {  
  const [state, dispatch] = useReducer(reducer, createInitialState(username))  
  // ...  
}
```

Although the result of `createInitialState(username)` is only used for the initial render, you're still calling this function on every render. This can be wasteful if it's creating large arrays or performing expensive calculations.

To solve this, you may **pass it as an *initializer* function** to `useReducer` as the third argument instead:

```
function createInitialState(username) {  
  // ...  
}  
  
function TodoList({ username }) {  
  const [state, dispatch] = useReducer(reducer, username, createInitialState)  
  // ...  
}
```

Notice that you're passing `createInitialState`, which is the *function itself*, and not `createInitialState()`, which is the result of calling it. This way, the initial state does not get re-created after initialization.

In the above example, `createInitialState` takes a `username` argument. If your initializer doesn't need any information to compute the initial state, you may pass `null` as the second argument to `useReducer`.

The difference between passing an initializer and passing the initial state directly

1. Passing the initializer function 2. Passing the initial state directly



Example 1 of 2:

Passing the initializer function

This example passes the initializer function, so the `createInitialState` function only runs during initialization. It does not run when component re-renders, such as when you type into the input.

TodoList.js

```
import { useReducer } from 'react';

function createInitialState(username) {
  const initialTodos = [];
  for (let i = 0; i < 50; i++) {
    initialTodos.push({
      id: i,
      text: username + "'s task #" + (i + 1)
    });
  }
  return {
    draft: '',

```

▼ Show more

Next Example

Troubleshooting

I've dispatched an action, but logging gives me the old state value

Calling the `dispatch` function **does not change state in the running code**:

```
function handleClick() {  
  console.log(state.age); // 42  
  
  dispatch({ type: 'incremented_age' }); // Request a re-render with 43  
  console.log(state.age); // Still 42!  
  
  setTimeout(() => {  
    console.log(state.age); // Also 42!  
  }, 5000);  
}
```

This is because [states behaves like a snapshot](#). Updating state requests another render with the new state value, but does not affect the `state` JavaScript variable in your already-running event handler.

If you need to guess the next state value, you can calculate it manually by calling the reducer yourself:

```
const action = { type: 'incremented_age' };
dispatch(action);

const nextState = reducer(state, action);
console.log(state);    // { age: 42 }
console.log(nextState); // { age: 43 }
```

I've dispatched an action, but the screen doesn't update

React will ignore your update if the next state is equal to the previous state, as determined by an `Object.is` comparison. This usually happens when you change an object or an array in state directly:

```
function reducer(state, action) {
  switch (action.type) {
    case 'incremented_age': {
      // 🚩 Wrong: mutating existing object
      state.age++;
      return state;
    }
    case 'changed_name': {
      // 🚩 Wrong: mutating existing object
      state.name = action.nextName;
      return state;
    }
    // ...
  }
}
```

You mutated an existing `state` object and returned it, so React ignored the update. To fix this, you need to ensure that you're always [updating objects in](#)

state and updating arrays in state instead of mutating them:

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'incremented_age': {  
      // ✅ Correct: creating a new object  
      return {  
        ...state,  
        age: state.age + 1  
      };  
    }  
    case 'changed_name': {  
      // ✅ Correct: creating a new object  
      return {  
        ...state,  
        name: action.nextName  
      };  
    }  
    // ...  
  }  
}
```

A part of my reducer state becomes undefined after dispatching

Make sure that every case branch **copies all of the existing fields** when returning the new state:

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'incremented_age': {  
      return {  
        ...state, // Don't forget this!  
        age: state.age + 1  
      };  
    }  
  }  
}
```

Without `...state` above, the returned next state would only contain the `age` field and nothing else.

My entire reducer state becomes undefined after dispatching

If your state unexpectedly becomes `undefined`, you're likely forgetting to `return state` in one of the cases, or your action type doesn't match any of the `case` statements. To find why, throw an error outside the `switch`:

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'incremented_age': {  
      // ...  
    }  
    case 'edited_name': {  
      // ...  
    }  
  }  
  throw Error('Unknown action: ' + action.type);  
}
```

You can also use a static type checker like TypeScript to catch such mistakes.

I'm getting an error: “Too many re-renders”

You might get an error that says: Too many re-renders. React limits the number of renders to prevent an infinite loop. Typically, this means that you're unconditionally dispatching an action *during render*, so your component enters a loop: render, dispatch (which causes a render), render, dispatch (which causes a render), and so on. Very often, this is caused by a mistake in specifying an event handler:


```
// 🚩 Wrong: calls the handler during render
return <button onClick={handleClick()}>Click me</button>

// ✅ Correct: passes down the event handler
return <button onClick={handleClick}>Click me</button>

// ✅ Correct: passes down an inline function
return <button onClick={(e) => handleClick(e)}>Click me</button>
```

If you can't find the cause of this error, click on the arrow next to the error in the console and look through the JavaScript stack to find the specific `dispatch` function call responsible for the error.

My reducer or initializer function runs twice

En el [modo estricto](#), React llamará a tus funciones reductora e inicializadora dos veces. Esto no debería provocar errores en tu código.

Este comportamiento **exclusivo para desarrollo** te ayuda a [mantener la pureza de los componentes](#). React utiliza el resultado de una de las llamadas e ignora el de la otra. Siempre que tus funciones de componente, inicializador y reductor sean puras, esto no debería afectar tu lógica. Sin embargo, si resultan ser impuras por accidente, esto te ayuda a detectar los errores.

Por ejemplo, esta función reductora impura modifica un array en estado:

```
function reducer(state, action) {
  switch (action.type) {
    case 'added_todo': {
      // 🚩 Mistake: mutating state
      state.todos.push({ id: nextId++, text: action.text });
      return state;
    }
    // ...
  }
}
```

Debido a que React llama a tu función reductora dos veces, verás que la tarea se agregó dos veces, por lo que sabrás que hay un error. En este ejemplo, puedes corregir el error reemplazando [el array en lugar de modificarlo](#) :

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'added_todo': {  
      // ✅ Correct: replacing with new state  
      return {  
        ...state,  
        todos: [  
          ...state.todos,  
          { id: nextId++, text: action.text }  
        ]  
      };  
    }  
    // ...  
  }  
}
```

Ahora que esta función reductora es pura, llamarla una vez más no afecta su comportamiento. Por eso, el hecho de que React la llame dos veces ayuda a detectar errores. **Solo las funciones de componente, inicializador y reductor deben ser puras.** Los manejadores de eventos no necesitan serlo, por lo que React nunca los llamará dos veces.

Para obtener más información, lea el artículo [sobre cómo mantener los componentes puros](#) .

< ANTERIOR
[usar optimista](#)

PRÓXIMO >
[useRef](#)

Meta Open Source

Copyright © Meta Platforms, Inc

¿uwu?

Aprende React

Inicio rápido

Instalación

Descripción de la interfaz de usuario

Agregar interactividad

Gestión del Estado

escotillas de escape

Referencia de la API

API de React

APIs DOM de React

Comunidad

Código de conducta

Conozca al equipo

Colaboradores de la documentación

Expresiones de gratitud

Más

Blog

React Native

Privacidad

Términos

